

Job Application Tracker – Architecture & Codebase Documentation

1. Overview

A full-stack web application that lets users register/login, create and manage job applications, and track them in multiple visual formats (cards, table, and kanban). A daily reminder job can email follow-up reminders based on each application's follow-up date. The system is split into a React + Vite front end and a Node.js + Express + MongoDB back end with JWT-based authentication. The client communicates with the server via REST APIs using Axios, and the server persists data in MongoDB using Mongoose.

2. Architecture at a Glance

High-Level Components

- **Client (React + Vite)**
 - Renders authentication pages (Login/Register) and a protected dashboard.
 - Shows applications in multiple views (Cards/Table/Kanban).
 - Makes API calls to the server and manages auth state via context + localStorage.
- **Server (Node.js + Express + MongoDB)**
 - Exposes REST endpoints for authentication and application CRUD.
 - Secures protected routes with JWT tokens (Bearer header or HttpOnly cookie).
 - Runs a daily cron job to email follow-up reminders.

Data Flow (Request Lifecycle)

1. **User registers or logs in** via client form → `POST /api/auth/register` or `POST /api/auth/login`.
2. **Server validates credentials**, generates JWT, and stores it in an HttpOnly cookie.
3. **Client fetches applications** → `GET /api/applications` (with cookie or Bearer token) and renders views.
4. **Updates/Deletes** are sent via `PUT /api/applications/:id` or `DELETE /api/applications/:id`.
5. **Daily cron job** checks for applications with follow-up dates and emails reminders.

3. Tools & Technologies

Front End

- **React 19** – UI component model.
- **React Router 7** – Routing for login, register, and dashboard.
- **Vite** – Development server and build tool.
- **Tailwind CSS (v4)** – Utility-first styling using CSS-in-file @utility definitions.
- **GSAP** – UI animations for auth screens and modal transitions.
- **Dnd-Kit** – Drag-and-drop for the Kanban view.
- **Axios** – HTTP client for API calls.

Back End

- **Node.js + Express 5** – HTTP server and REST API.
- **MongoDB + Mongoose** – Data modeling and persistence.
- **JWT (jsonwebtoken)** – Authentication tokens.
- **bcrypt** – Password hashing.
- **Nodemailer** – Email reminders for follow-ups.
- **node-cron** – Scheduled reminder job.
- **Helmet / CORS / cookie-parser / rate-limit** – Security and request handling.

Dev Tooling

- **ESLint** – Linting for React code.
- **Nodemon** – Live reload for the server.

4. How the Site Works (End-to-End)

1. Authentication

- Users create an account or log in. The server returns a session cookie containing the JWT token.
- The client stores a minimal auth state in `localStorage` to control routing and access to protected pages.

2. Dashboard

- After authentication, users land on the **Dashboard** which displays statistics and application data.
- Users can filter applications by status and choose between **Cards**, **Table**, or **Kanban** layouts.

3. Application Management

- Users create a new application via a modal form.

- Editing happens in a modal details view, allowing updates to status, follow-up date, notes, etc.
- Users can delete entries in the card view.

4. Follow-Up Reminders

- A cron job runs daily at 9 AM to find applications due for follow-up.
- Reminder emails are sent to the user's registered email and then marked as sent.

5. Repository Structure & File-by-File Guide

Client (/client)

- `package.json` – Front-end dependencies and scripts.
- `package-lock.json` – Locked dependency tree for reproducible installs.
- `vite.config.js` – Vite configuration with React + Tailwind plugins.
- `eslint.config.js` – ESLint configuration for React code and hooks.
- `index.html` – Root HTML shell that mounts the React app.
- `public/auth-bg.png` – Background image for auth pages.
- `src/main.jsx` – React entry point that mounts the app and wraps providers.
- `src/App.jsx` – Root component that renders routes.
- `src/index.css` – Tailwind + custom utilities + theme definitions.

Client API Layer (/client/src/api)

- `axios.js` – Axios instance with base URL and cookie support.
- `auth.api.js` – API helpers for register/login/logout.
- `applications.api.js` – API helpers for CRUD operations on applications.

Client Context (/client/src/context)

- `AuthContext.jsx` – Authentication state container with login/logout helpers.

Client Components (/client/src/components)

- `Navbar.jsx` – Top navigation bar with logout action.
- `ProtectedRoute.jsx` – Guards routes for authenticated users only.
- `Stats.jsx` – Displays application summary statistics.
- `ApplicationForm.jsx` – Form to create a new application.
- `ApplicationCard.jsx` – Card view of an application with view/delete actions.
- `ApplicationDetailsModal.jsx` – Modal to view/edit application details.

Client Pages (/client/src/pages)

- `Login.jsx` – Login screen with animated UI.
- `Register.jsx` – Registration screen with animated UI.
- `Dashboard.jsx` – Wraps the main application area and navigation.
- `Applications.jsx` – Core dashboard logic for loading, filtering, and rendering views.

Client Views (/client/src/views)

- `ViewTabs.jsx` – Toggle between cards/table/kanban views.
- `CardView.jsx` – Grid of application cards.
- `TableView.jsx` – Table layout with inline status editing.
- `KanbanView.jsx` – Drag-and-drop kanban board.
- `KanbanColumn.jsx` – Kanban column container (Applied/Interviewing/Offer/Rejected).
- `KanbanCard.jsx` – Individual draggable card in kanban view.

Client Constants (/client/constants)

- `statuses.js` – Allowed application statuses and associated color classes.

Server (/server)

- `package.json` – Server dependencies and scripts.
- `package-lock.json` – Locked dependency tree for reproducible installs.
- `server.js` – Server entry point: loads env vars, connects DB, starts cron job, listens on port.
- `app.js` – Express app configuration (middleware, CORS, routes).

Server Config (/server/config)

- `db.js` – MongoDB connection via Mongoose.
- `mailer.js` – Nodemailer transport configuration (Gmail-based).

Server Controllers (/server/controllers)

- `auth.controller.js` – Register, login, logout with JWT and cookies.
- `application.controller.js` – CRUD logic for job applications.

Server Models (/server/models)

- `User.js` – User schema (email + password).
- `Application.js` – Application schema (company, role, status, follow-up date, notes, etc.).

Server Routes (/server/routes)

- `auth.routes.js` – Auth endpoints wiring to controller.
- `application.routes.js` – Application endpoints protected by auth middleware.

Server Middleware (/server/middleware)

- `auth.middleware.js` – JWT-based route protection using cookies or Bearer tokens.
- `validateObjectId.js` – Validates MongoDB ObjectId parameters.

Server Jobs (/server/jobs)

- `reminder.job.js` – Daily cron job that sends follow-up reminder emails.

Server Dependencies (/server/node_modules , /client/node_modules)

- Installed third-party packages. These are generated by package managers and should not be edited manually.